

Foundations and the AI service layer

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build `nw/llm`: the one client every later project calls a model through, behind the provider switch and the model gateway. Bounded concurrency, provider-aware retries, a fallback and a breaker, validated structured output, ordered fan-out, a hard spend cap, and a correlation ID on every line.

AI, machine learning, deep learning, generative AI and agents nest inside each other

Artificial intelligence infers from its input how to produce predictions, content, recommendations or decisions

Machine learning learns from data instead of hand-written rules

PROJECT 1

Deep learning layered neural networks learn their own representations

PROJECT 2

Generative AI generates new content shaped like its training data

PROJECT 3

Agentic AI a model that plans and acts through tools, in a loop

PROJECT 4

The service layer, today, is how every ring that calls a model reaches it. The capstone runs all four rings as one system.

Since 2025, four shifts changed what you build

Models reason before they answer

DeepSeek-R1 showed that reinforcement learning alone can teach an LLM to reason, without human-written reasoning traces. January 2025.

For you: thinking time is a setting, and you pay for it.

Context windows grew about 30x a year

Since 2023, by Epoch AI's estimate. One-million-token windows ship today, closed and open.

For you: long context now competes with retrieval.

Capability got about 13x cheaper a year

The price of reaching a fixed level of performance, since 2023, by Epoch AI's estimate.

For you: a design too expensive today may fit next year.

Agents reached production

40 percent of respondents at firms above 1 billion USD revenue report scaling AI agents, up from 27 percent a year earlier.

For you: approvals and limits are now production work.

The frontier changes by the quarter, and open weights trail it by months

Since January 2026 the best open-weight models have trailed the best closed ones by about four months.

Lab	Latest release, September 2026	Weights
Anthropic	Claude Opus 5.5, 22 September	closed
OpenAI	the GPT - 5.6 family, July and August	closed
Google	Gemini 3.6 Flash, July; Gemma 4, April	closed; Gemma open, Apache 2.0
Meta	Muse Spark, April, replacing open Llama	proprietary
DeepSeek	V4, April, a one - million - token window	open, MIT
Alibaba, Moonshot, Zhipu	Qwen 3.5 and 3.6, Kimi K2.6, GLM - 5	open

This table will be wrong within a quarter. That is why your code names a role and configuration names the model.

Most companies use AI, and few yet earn money from it

88%

of organisations regularly use AI
(2025)

37%

report any EBIT contribution from
AI, unchanged in a year (2026)

6%

are high performers, where AI
moves the bottom line; the share
stays near 6 percent (2026)

The gap between the first number and the other two is engineering: evaluation, cost, operations and a person who approves. It is what the six parts build.

Every capability you ship arrives with a failure you own

The AI Index counts 362 documented AI incidents in 2025, up from 233 the year before.

Where	The failure	The guard you build
Service layer	runaway spend, retry storms	a hard spend cap, a breaker
Project 1	a missed emergency, silent drift	a gate, a drift alarm, a person who approves
Project 2	a compressed model drops the rare class	an int8 bar, a model card
Project 3	invented answers, prompt injection	citations or a refusal, a calibrated judge
Project 4	an agent acting beyond its brief	least privilege, human approval, a kill switch
Capstone	a bad release reaches customers	a canary, an alarm, a rollback

Eight roles share the work, and each owns a different part

Role	Owns	Where you practise it
AI engineer	model-backed features end to end: service, evaluation, cost	every part
ML engineer	models trained from data, validated and served	Projects 1 and 2
GenAI engineer	retrieval, prompts, grounding, output evaluation	Project 3
Agentic AI engineer	tools, orchestration, approvals, trajectories	Project 4
MLOps and platform engineer	pipelines, registries, gateways, observability	the platform, the capstone
Applied scientist	new methods and the experiments that justify them	the Project 2 benchmark
AI product manager	the problem, the metric, what good enough means	the gate's bars
AI safety and governance	risk classification, policy, audit, red-teaming	the capstone

Most AI engineers came from software, and the ladder above them is the same one

How people arrive

LinkedIn ranks AI engineer the fastest-growing US job of 2026.

- Top feeder roles: software engineer, data scientist, full-stack engineer
- Median prior experience: 3.7 years
- Top skills: LangChain, RAG, PyTorch

Track	Levels, one company's example
Individual contributor	IC1 to IC4 engineer, IC5 Staff, IC6 Principal, IC7 Senior Principal
Manager	M3 to M7, up to Senior Engineering Director

Levels differ by scope, reach and levers for impact, not by the model you use. Side moves are common: ML engineer to platform, AI engineer to agentic work or product, any engineer to governance.

Northwind's support desk is slow where it must be fast

About 8,000 tickets a quarter

From B2B customers, in English and German. About one in twenty is German.

3.9 percent are P0

Outages and security incidents. The Enterprise P0 SLA is 15 minutes.

Read in the order they arrive

A P0 waits behind a question about dark mode until a person reaches it.

Your brief as Northwind's new AI engineer: make the support desk faster without making it dangerous.

Six parts build one system, with the right kind of AI for each job

Part	Opens with	You build
TODAY The service layer	the AI landscape and your place in it	one client and one gateway for every model call
Project 1	classical ML still earns its place	a triage model, a pipeline, a registry, a person who approves
Project 2	deep learning and the transformer	a fine-tuned encoder, exported, quantized and gated
Project 3	LLMs in production	grounded answers with citations or a refusal, judged and gated
Project 4	agentic AI	a resolver agent with tools, approvals and a tiered gate
Capstone	from a system to an organisation	promotion, delivery, SLOs, cost per resolved ticket

Today you build the service layer every model call goes through

The callers

Project 3's policy answers,
Project 4's agent, the capstone's
router and its judge.

`nw/llm`, today

Roles, bounded concurrency,
retries, a fallback and a breaker,
validated output, a spend cap,
one correlation ID.

The model gateway

One door per track, your key
and your budget, a guardrail in
front of the Workhorse, the
Economy model and the Judge.

Solve the hard parts of calling a model once, here, so no project solves them again.

Today, step by step

- 1 Confirm the track, then switch to the fake provider
- 2 Read the contract and see the tests fail
- 3 Backoff with jitter, honouring `Retry-After`
- 4 Complete with a semaphore, retries and metering
- 5 Structured output with one repair round trip
- 6 Fan out and keep the order
- 7 Install the skill set and set the review convention
- 8 One live call on the Workhorse
- 9 The model gateway, your key and what sits behind it

How the room works

Steps 2 to 6 run on the fake provider: no network, no spend, outcomes a real provider will not produce on demand.

Red tests are the spec; green is the proof. Your coding agent writes alongside you from step 7, held to the same bar.

Three route lines decide where every call goes

Setting	Workhorse and Economy	Judge
<code>NW_PROVIDER=fake</code>	the fake provider, in memory	the fake provider
<code>NW_GATEWAY_URL</code> set	the model gateway, your key as bearer	the gateway
AWS, direct	Bedrock Converse	Anthropic SDK, Bedrock client
Google Cloud, direct	the managed API, OpenAI-compatible	Anthropic SDK, Vertex client
Local, direct	Ollama	fake judge, or the Anthropic API with a key

`make_provider` is the only place a track name turns into a vendor client. `make_preflight` prints its decision as `route <role>`, from `.env` alone, with no network.

Code names roles, configuration names models

Role	AWS	Google Cloud	Local
Workhorse	gpt-oss-120b	gpt-oss-120b	<code>gpt-oss:20b</code> on Ollama
Economy	Nova Micro	gpt-oss-120b with the Economy caps	<code>gpt-oss:20b</code>
Judge	Claude Opus 5	Claude Opus 5	Claude through a key, or judge-free

Two roles are open-weight, the third is the calibrated judge: a judge should be stronger than the model it grades. `ModelRole.WORKHORSE` in code, the id in `.env`; nothing outside `nw/llm/providers/` imports a vendor SDK.

Every project hits the same three walls

Fan-out

500 prompts, 500 open connections, then a wall of 429s.

Bounded concurrency.

Unreliable upstream

The batch dies at prompt 312 and everything before it is lost.

Provider-aware retries.

Unvalidated output

`json.loads` raises on a reply that begins "Sure, here is".

Structured output with repair.

Solve them once, in one client. `make session01` on the skeleton: 34 failed, 92 passed. The 92 are the given layers; the 34 are `retry.py` and four methods in `client.py`.

A model call is waiting, not computing

Hundreds of milliseconds to several seconds on the network, almost no local work. That decides the design:

- `asyncio`, not threads: idle waiting costs nothing
- one `httpx` pool per process
- a semaphore sized to the provider's limit

The semaphore is a business number

It comes from the quota and from how many services share it, not from the core count.

Two services, one quota of 50 a minute: neither gets 50.

Retryable and terminal are different failures

Status or error	Family	Why
429, 529, 5xx, timeout	retryable	the provider's or the network's problem, and transient
400, 401, 403, 404	terminal	your request; it fails the same way again
refusal stop reason	terminal	the content, not the call

Retrying a 400 spends money on a certain failure. Not retrying a 429 loses work that would have succeeded a second later.

Backoff needs jitter, or the spike comes back

```
ceiling = min(base * 2 ** (attempt - 1), cap)
delay = random.uniform(0, ceiling)
if retry_after is not None:
    delay = min(retry_after, retry_after_cap)
```

- The ceiling doubles per attempt; the delay is uniform below it
- Without jitter, every client that failed together retries together
- `Retry-After` overrides everything: the provider chose that number

When the model itself is the outage: fall back, then stop asking

- **Fallback:** `NW_MODEL_FALLBACK_<ROLE>` names a second model per role, tried once when the primary is exhausted or gone, never for a refusal
- **Circuit breaker:** after `NW_BREAKER_FAILURES` failures in a row a model is skipped for `NW_BREAKER_OPEN_S` seconds
- **Timeout:** `NW_REQUEST_TIMEOUT_S` turns a provider that never answers into a retryable timeout

Every completion says how it was obtained

The span and the log line carry the model that answered, `fallback`, `attempts` and the models the breaker skipped. The cost record names the model that answered, not the one you asked for.

Structured output is a contract, not a hope

- 1 Tell the model the JSON Schema of the Pydantic class you want
- 2 Parse, then validate. Never trust the formatting
- 3 On failure, send the error back once, with the reply it made
- 4 Second failure: raise. A loop that keeps asking is a budget leak

This matters more on an open-weight Workhorse than it did on a frontier model: validation in your own code is what survives a model change, and the repair round trip is where a new model's formatting habits show first.

Every call leaves one record under one key

- The meter **reserves** an estimate inside the semaphore, before the call, and replaces it with the real cost after
- The cap is a hard stop that refuses the call, not a dashboard that notices later
- The span carries the model, tokens, cost, fallback, attempts and `nw.prompt_version`
- `map(..., max_total_tokens=N)` stops sending once the fan-out has spent its tokens

Trace, log, bill: one key

When support asks what happened to ticket T-200005, you search one correlation ID and get the model calls, their cost, the prompt version and the spans in the order they ran.

The gateway is one door, and your key is your budget

The gateway holds	The client keeps
one virtual key per tenant	roles
a budget per 30 days	the breaker and the fallback
the models the key may call	the spend cap per process
the price of every call, per tenant	the token budget per fan-out
the guardrail in front of the model	the correlation ID

Two layers, two owners

The gateway protects the platform's bill and content. The client protects the caller's latency and its own result.

A gateway that is down looks to the client exactly like a provider that is down.

One live call, then prove the fallback through the gateway

```
uv run python scripts/live_check.py

NW_MODEL_WORKHORSE=does-not-exist \
NW_MODEL_FALLBACK_WORKHORSE=<Economy id> \
  uv run python scripts/live_check.py
```

- The first run: one JSON line with `correlation_id: live-check`, the Workhorse model, latency, tokens and cost
- The second: the gateway answers 400 or 404 naming `does-not-exist`, the client does not retry a terminal error, and the line reads the Economy model with `fallback: true`
- One bad call does not open the breaker; three in a row would

What exists after today, AWS track



The nw package and the toolchain on your laptop; on the platform, the model gateway in front of gpt-oss and Claude on Bedrock, and your key in Secrets Manager. Ringed: what this part adds.

What exists after today, Google Cloud track



Same package, same toolchain; the gateway on Cloud Run in front of gpt-oss and Claude on the Agent Platform (formerly Vertex AI), your key in Secret Manager.

What exists after today, Local track



Same package; the gateway is LiteLLM on :4000 in the compose stack, gpt-oss on Ollama behind it, your key a line in `tenants.tsv`.

Same client, three platforms behind it

Component	AWS	Google Cloud	Local
Model gateway	LiteLLM on ECS Fargate	LiteLLM on Cloud Run	LiteLLM on :4000
Your key	Secrets Manager	Secret Manager	<code>deploy/local/litellm/tenants.tsv</code>
Workhorse	gpt-oss-120b on Bedrock	gpt-oss-120b, managed API	<code>gpt-oss:20b</code> on Ollama
Guardrail in the gateway	Bedrock Guardrails	Model Armor	Llama Guard 3 through Ollama
Traces	X-Ray with Transaction Search	Cloud Trace	OpenTelemetry collector, Jaeger

Nothing was deployed today. The client runs on your laptop against the platform's gateway.

Agent-written code, same bar

`/tdd`

Start from the failing test, not from the prompt.

`/code-review`

Before you run the suite, not after it is green.

`/session-check`

Reads the acceptance criteria against your checkout and reports each one.

Nothing enters a project because an agent produced it. It passes the tests or it does not go in.

Acceptance criteria

1. 100 prompts at `max_concurrency=8` never exceed 8 in flight
2. Two 429s then success: succeeds. A 400: fails at once, one attempt
3. A dead model falls back once and the record names who answered
4. `structured` recovers from one malformed reply and raises after the second
5. `map` preserves order; spend and tokens are capped before the call

126 passed

what `make session01` prints when you are done

Questions worth asking

- What is the right concurrency for a provider that allows 50 requests a minute, shared by two services?
- When is the repair round trip more expensive than failing fast?
- The gateway has a budget per key and the client a cap per process. Which do you want to hit first, and who gets told?
- Which of today's failures would have reached production in your current setup?